Role-based FileSystem and GitHub Access for LLM Agents

Vamshi Sunku Mohan*[1]

[1] Center for Cybersecurity Systems and Networks, Amrita Vishwa Vidyapeetham, Amritapuri.
*corresponding author: vamshisunkumohan@gmail.com

Keywords: Agentic AI, MCP, GitHub, FileSystem.

Objective. This project develops an MCP agent connecting users to GitHub and local file systems with access defined based on the roles performed. In particular, we define users belonging to 3 different roles being, Engineering, IT and Sales and designate read-write permissions based on the individual jobs.

1 System Design

To enable a role-based access control to GitHub and local File System, we define personas, i.e., Engineer, IT and Sales with different job descriptions and located in different countries around the world. We then define tasks per persona as in Table 1 and map tasks to GitHub repository and File System, which in turn reads, updates and retrieves information via GitHub MCP. Pre-processing tasks to map user prompts to task execution are defined in Table 2,

Persona	Permission Type	Description
Engineering (eng01)	GitHub Read	Allowed
	GitHub Write	Allowed
	File System Access	Engineering folder only
	Ticket Create	Allowed
	Ticket Update	Not Allowed
	Ticket Close	Not Allowed
IT (it01)	GitHub Read	Allowed
	GitHub Write	Not Allowed
	File System Access	IT folder only
	Ticket Create	Not Allowed
	Ticket Update	Allowed
	Ticket Close	Allowed
Sales (sales01)	GitHub Read	Not Allowed
	GitHub Write	Not Allowed
	File System Access	Sales folder only
	Ticket Create	Allowed
	Ticket Update	Not Allowed
	Ticket Close	Not Allowed

Table 1: Role-Based Permission Matrix for Personas

2 Core Components

Core components of the system contains,

- User - Operating using interactive CLI to connect to application layer.
- Application layer - Defined in main.py in the code files. This layer parses the prompt based on occurrence of the words in historically-used prompts. Tasks are then mapped and tools such as direct file access or MCP call are determined. hey are in turn mapped to MCP client layer.
- MCP Client layer - Defines persona per user role, checks permissions and determines the option to be performed based on parsed prompt text.

Component	Feature	Description
Prompt-to-Task Mapper	Keyword-Based Matching	Identifies tasks based on keywords found in prompts
	Multi-Task Detection	Supports extracting multiple task intents in one prompt
	Historical Prompt Pairs	Learns mappings from previously observed prompt-task examples
Persona-Task Association	Task-to-Persona Mapping	Each task is tied to a specific persona's job role
	Tool Requirements	Determines whether GitHub or FileSystem is needed for the task
	Persona Validation	Ensures the acting user matches the persona allowed for the task
Permission Engine	Matrix-Based Model	Centralized permission reference defining allowed actions
	Granular Control	Permissions vary by persona, tool, and operation
	Action Types	Supports READ and WRITE access enforcement
Tool Call Inference	Tool Selection	Determines which tools must be invoked from the prompt
	Action Inference	Infers whether the user intends READ or WRITE operations
	Resource Extraction	Identifies repository paths or file system targets from prompt text

Table 2: Core System Components and Functional Responsibilities

- Docker MCP Client - Deploys local docker images of GiHub MCP server
- Apps - Apps defined in this project are GitHub connecting Engineers via MCP, file systems and SqLite database to store and track ticket status.

Some of the functions defined in code are listed as follows in Table 3,

Layer	Function	Description
Application layer	process_prompt	Parses user input and maps to tasks
	_determine_tools	Decides which tools to use (GitHub/FileSystem)
	_execute_tools	Runs operations on selected tools
	display_results	Prints results
MCP Client layer	github_list_repos	Lists repositories
	github_get_file, github_create_file	Retrieves and creates file in specified GitHub repositories
	filesystem_read_file, filesystem_write_file	Reads and write specified local files
	create_ticket, list_tickets, update_ticket	Creates, lists and updates tickets
Docker MCP Client	start	initializes Docker container with GitHub MCP server
	call$_{t}ool$	Invokes MCP tools via JSON-RPC
	list_repositories	Searches for user's repositories

Table 3: Core Components of Role-based access fro LLM Agents

3 Ticket Generation and Database Storage

As a part of IT department's job description, ticket handling is defined. In his project, Employees from Engineering and Sales teams are permitted to raise and view tickets. IT team employees are permitted to view the tickets, update status and close them individually. To ensure tickets are stored and addressed structurally, we use a SQLite which is a structured database. Tickets are stored, updated and accessed using SQL queries. Information stored in database are shown in Table 4,

4 Integration with MCP

Start the local docker image of GitHub server using "ghcr.io/github/github-mcp-server" to enable GitHub MCP server and set the authentication token in ".env" file and pass the token as a local variable. Handshake procedure to authenticate involves the MCP client sending "initialize" request via JSON-RPC over stdin. Server responds with capabilities and available tools with which the client can execute operations.

5 Fallback Strategy

The system switches to use GitHub REST API using httpx library if Docker or MCP fails due to container exit, timeout or connection error.

Table Name	Feature	Description
Ticket Storage Table	ticket_id	Format: TKT-YYYYMMDDHHMMSS-persona_id
	title	Ticket title
	description	Detailed issue description
	status	Open, Resolved, Closed
	persona_id	Creator's persona ID (eng01, sales01)
	persona_name	Employee's full name
	team	Employee's team (Engineering, Sales)
	priority	Low, Medium, High
	category	Optional query classification
	created_at, updated_at, resolved_at	Timestamps indicating when tickets were created, updated and resolved
Ticket Updates Table	ticket$_id$	Links to tickets generation table
	update$_text$	Description of the change
	updated$_by$	Name of person who made the update
	updated$_at$	Timestamp of the update

Table 4: Ticket Generation and Updation Fields

6 Test Cases

Test cases implemented for each of the persona based on the permissions are listed in Table 5,

Persona	Function	Description
Engineering	List repositories	See all public and private repositories
	Read file	Get code content from GitHub
	Update file	Fix bug in code
	Raise ticket	Report infrastructure issue to IT
IT	List tickets	View all open and resolved tickets from all teams
	List open tickets	Views only open tickets
	List resolved tickets	Views only resolved tickets
	List repositories	Views GitHub repositories in read-only mode
	Read ticket	Reads the recent ticket and shows details
	Read ticket TKT-20250115143022-persona_ID	Reads the ticket specified and shows details
	Update/Modify ticket	Updates ticket content, status and priority
	Close/Resolve ticket	Updates status of most recent ticket to resolved and closes it
Sales	List files	List files in File system
	Read files	Reads files in File system
	Create files	Creates files in File system
	Raise ticket	Report infrastructure issue to IT

Table 5: Test Cases Implemented per persona